We can build the ReasoningEngine, which will:

- generate a plan
- execute steps
- call the provider
- parse tool calls
- invoke tools
- log StepResults
- return final output

This is the core of the Intelligence Layer.

If you're ready, I'll generate:

📘 rca_intelligence/engine.py
The full ReasoningEngine implementation.

```
from __future__ import annotations

import json
from typing import Any, Dict, List, Optional, Callable
from dataclasses import dataclass, field

from rca_intelligence.models import (
    ReasoningContext,
    ToolCall,
    StepResult,
    StrategyPlan,
    StrategyStep,
)
from rca_intelligence.providers import ProviderAgent

# ---------------------------------------------------------
# Tool Registry
# ---------------------------------------------------------
@dataclass
class ToolRegistry:
    tools: Dict[str, Callable[..., Any]] = field(default_factory=dict)

    def register(self, name: str, fn: Callable[..., Any]) -> None:
        self.tools[name] = fn
```

```
    def get(self, name: str) -> Callable[..., Any]:
        if name not in self.tools:
            raise KeyError(f"Tool '{name}' not registered")
        return self.tools[name]

—----------

# ------------------------------------------------------------
# ReasoningEngine
# ------------------------------------------------------------
@dataclass
class ReasoningEngine:
    provider: ProviderAgent
    tools: ToolRegistry

    async def run(
        self,
        ctx: ReasoningContext,
        goal: str,
        plan: Optional[StrategyPlan] = None,
        *,
        temperature: float = 0.2,
        max_tokens: int = 2048,
    ) -> StepResult:
        """
        Executes a reasoning session:
        - If no plan is provided, ask the model to create one.
        - Execute each step.
        - Handle tool calls.
        - Return final StepResult.
        """

        # ------------------------------------------------
        # 1. If no plan, ask the model to generate one
        # ------------------------------------------------
        if plan is None:
            plan = await self._generate_plan(ctx, goal)

        step_results: List[StepResult] = []

        # ------------------------------------------------
        # 2. Execute each step in the plan
        # ------------------------------------------------
```

```
        for idx, step in enumerate(plan.steps):
            step_result = await self._execute_step(
                ctx=ctx,
                step=step,
                step_index=idx,
                temperature=temperature,
                max_tokens=max_tokens,
            )
            step_results.append(step_result)

        # ----------------------------------------------------
        # 3. Final output is the last step's output
        # ----------------------------------------------------
        return step_results[-1]
```

—--------

Plan Generation

```
async def _generate_plan(
        self,
        ctx: ReasoningContext,
        goal: str,
    ) -> StrategyPlan:

        prompt = (
            "You are the RCA ReasoningEngine. "
            "Create a JSON reasoning plan to achieve the following goal:\n"
            f"{goal}\n\n"
            "Return ONLY valid JSON with this structure:\n"
            "{ 'steps': [ { 'description': str, 'tool': str|null, 'arguments': object|null } ] }"
        )

        response = await self.provider.complete(
            ctx,
            prompt,
            json_mode=True,
        )

        data = json.loads(response["text"])
        steps = [
            StrategyStep(
```

```
            description=s["description"],
            tool=s.get("tool"),
            arguments=s.get("arguments"),
        )
        for s in data["steps"]
    ]

    return StrategyPlan(goal=goal, steps=steps)
```

—------

```
async def _execute_step(
    self,
    ctx: ReasoningContext,
    step: StrategyStep,
    step_index: int,
    *,
    temperature: float,
    max_tokens: int,
) -> StepResult:

    # ------------------------------------------------
    # 1. Build prompt for this step
    # ------------------------------------------------
    prompt = (
        f"Step {step_index}: {step.description}\n"
        f"Tool: {step.tool}\n"
        f"Arguments: {json.dumps(step.arguments or {})}\n\n"
        "If a tool is needed, return a tool call."
    )

    # ------------------------------------------------
    # 2. Call provider
    # ------------------------------------------------
    response = await self.provider.complete(
        ctx,
        prompt,
        temperature=temperature,
        max_tokens=max_tokens,
        tools=None,  # Tools are handled manually
    )

    text = response["text"]
```

```
        tool_calls = response["tool_calls"]

        # ------------------------------------------------
        # 3. Execute tool calls
        # ------------------------------------------------
        tool_results: Dict[str, Any] = {}

        for tc in tool_calls:
            tool_fn = self.tools.get(tc.name)
            result = await tool_fn(ctx=ctx, **tc.arguments)
            tool_results[tc.call_id] = result

        # ------------------------------------------------
        # 4. Return StepResult
        # ------------------------------------------------
        return StepResult(
            step_index=step_index,
            thought=text,
            tool_calls=tool_calls,
            tool_results=tool_results,
            output=text if not tool_calls else None,
        )
```

—---------

rca_intelligence/reasoner.py

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict, List, Optional

from rca_intelligence.models import ReasoningContext, StrategyPlan, StepResult
from rca_intelligence.engine import ReasoningEngine


@dataclass
class Reasoner:
    """
    High-level interface for agents (Research, Correction, Executive, etc.).
    Wraps the ReasoningEngine with common patterns: ask, summarize, classify, decide.
    """

    engine: ReasoningEngine
```

```
async def ask(
    self,
    ctx: ReasoningContext,
    goal: str,
    plan: Optional[StrategyPlan] = None,
    *,
    temperature: float = 0.2,
    max_tokens: int = 2048,
) -> StepResult:
    """
    Generic reasoning entrypoint.
    """
    return await self.engine.run(
        ctx=ctx,
        goal=goal,
        plan=plan,
        temperature=temperature,
        max_tokens=max_tokens,
    )

async def summarize(
    self,
    ctx: ReasoningContext,
    text: str,
    *,
    temperature: float = 0.1,
    max_tokens: int = 1024,
) -> str:
    """
    Summarize arbitrary text.
    """
    goal = (
        "Summarize the following content clearly and concisely for an expert audience. "
        "Preserve key technical details.\n\n"
        f"{text}"
    )
    result = await self.ask(
        ctx=ctx,
        goal=goal,
        temperature=temperature,
        max_tokens=max_tokens,
    )
    return result.output or result.thought
```

```
async def classify(
    self,
    ctx: ReasoningContext,
    text: str,
    labels: List[str],
    *,
    temperature: float = 0.0,
) -> Dict[str, Any]:
    """
    Classify text into one of the given labels.
    Returns a dict with { 'label': str, 'rationale': str }.
    """
    goal = (
        "Classify the following text into one of the provided labels. "
        "Return ONLY JSON with keys: label, rationale.\n\n"
        f"Labels: {labels}\n\n"
        f"Text:\n{text}"
    )
    result = await self.ask(
        ctx=ctx,
        goal=goal,
        temperature=temperature,
    )
    # Expect JSON in result.thought/output
    import json

    raw = result.output or result.thought
    return json.loads(raw)

async def decide(
    self,
    ctx: ReasoningContext,
    situation: str,
    options: List[str],
    *,
    temperature: float = 0.2,
) -> Dict[str, Any]:
    """
    Structured decision helper over an explicit action space.
    Returns { 'choice': str, 'rationale': str }.
    """
    goal = (
        "Given the situation and options, choose the best option and explain why. "
```

```
            "Return ONLY JSON with keys: choice, rationale.\n\n"
            f"Situation:\n{situation}\n\n"
            f"Options: {options}"
        )
        result = await self.ask(
            ctx=ctx,
            goal=goal,
            temperature=temperature,
        )
        import json

        raw = result.output or result.thought
        return json.loads(raw)
```

—----------